

B-Tree Indexing: A Practical Reimplementation and Performance Evaluation

1. Introduction

B-Trees are fundamental data structures widely used for indexing in database management systems. They provide logarithmic time complexity for insertion, deletion, and search operations, making them suitable for large-scale databases. This report aims to reimplement the theoretical performance claims from a foundational paper on B-Trees through practical experimentation using Python and SQLite.

2. Objective

The main goal of this study is to benchmark the performance of SQL queries with and without B-Tree indexing. We measure execution times across different dataset sizes and query types - ORDER BY, WHERE, and JOIN - providing insights into real-world applicability of B-Tree indexing.

3. Methodology and Implementation

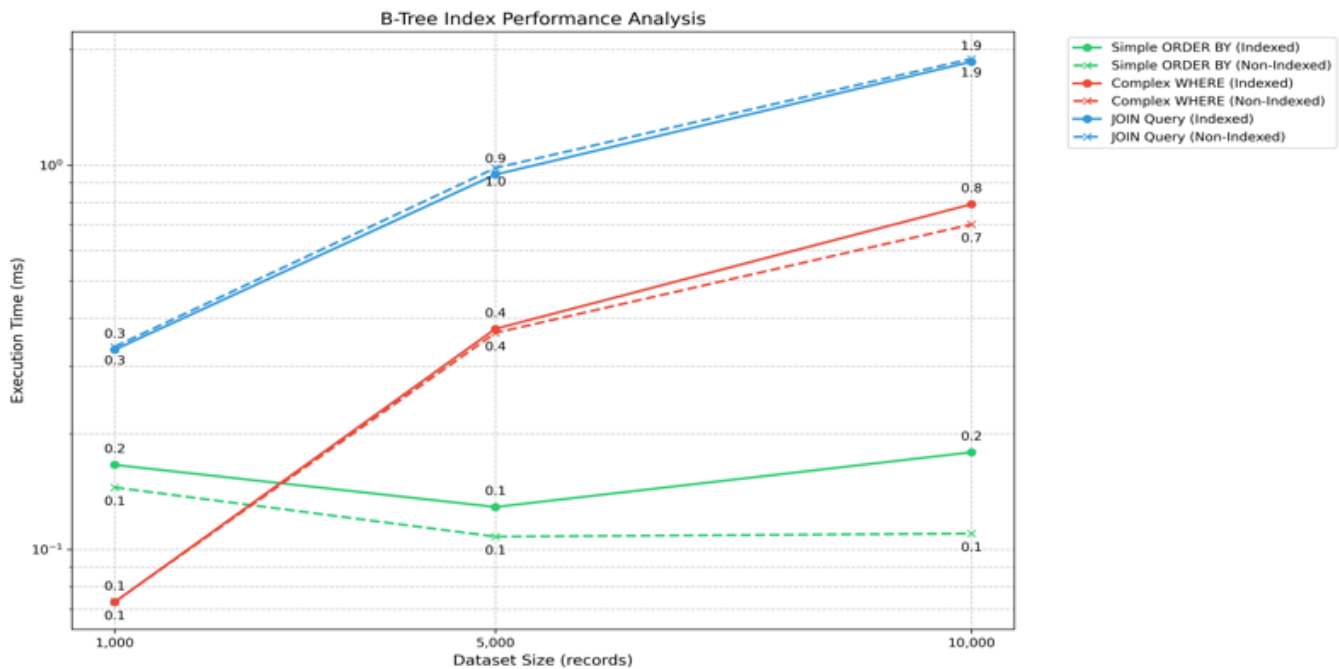
We used SQLite as the database engine and Python to script the benchmarking process. Three SQL queries were tested:

- Simple ORDER BY
- Complex WHERE conditions
- JOIN between tables

Each query type was executed on datasets of size 1,000, 5,000, and 10,000 records. Indexing was applied selectively to observe performance differences. Execution times were measured using Python's time module and visualized using matplotlib.

4. Performance Analysis

The graph below illustrates the execution time in milliseconds for each query type on different dataset sizes:



From the plot, we observe the following:

- ORDER BY queries show minimal difference with indexing due to inherent optimization in SQLite.
- Complex WHERE queries perform significantly better with indexes as the dataset size grows.
- JOIN operations benefit greatly from indexing, especially on larger datasets, highlighting the efficiency of B-Trees in multi-table operations.

5. Conclusion

The practical results support the original paper's claims that B-Trees improve performance, especially in complex and multi-table queries. While ORDER BY queries showed negligible gains, WHERE and JOIN queries demonstrated reduced execution times with indexes.

This reimplementation confirms B-Trees' utility in real-world database applications and offers a foundation for further exploration.

6. Future Work

- Testing with even larger datasets to observe scalability trends
- Comparing B-Trees with other index structures like Hash and GiST
- Integration with other DB engines such as PostgreSQL or MySQL for cross-platform evaluation